



KAIROS

`catch the moment code drifts from intent`

Intent-drift guardian and living docs, powered by IBM Bob

`kairos check`

`kairos fix`

`kairos handoff`

GitHub Action

The problem

Code drifts from intent, silently

- Code changes every day. **Specs, API contracts, READMEs and tests** rarely keep up.
- Linters check syntax. Tests check what someone remembered to test.
- **Nothing checks that a change still matches the intent**, until it becomes an incident, a broken onboarding or a wrong decision.
- With coding agents the code moves even faster, and every new chat starts without the history.

Kairos

An intent-drift guardian powered by IBM Bob



- Every finding has **evidence on both sides** (file:line), a severity and a proposed resolution.
- Kairos **does not assume the code is right**: for each finding, which side is the truth?
- Runs where drift should be caught: `kairos check`, pre-push hook, GitHub Action with a PR comment.

Demo: three ordinary commits

demo/orders-api: Express + TS, docs/SPEC.md, openapi.yaml, README, tests

```
$ kairos check
KRS-001 high SPEC_VIOLATION src/pricing.ts:8 discount 10% → 15%, SPEC not updated
KRS-002 high STALE_DOC src/config.ts:8-9 README + SPEC still say DB_URL
KRS-003 medium UNDOCUMENTED_BEHAVIOR src/routes/orders.ts DELETE /orders/:id not in OpenAPI
KRS-004 medium MISSING_TEST src/routes/orders.ts DELETE /orders/:id has no test

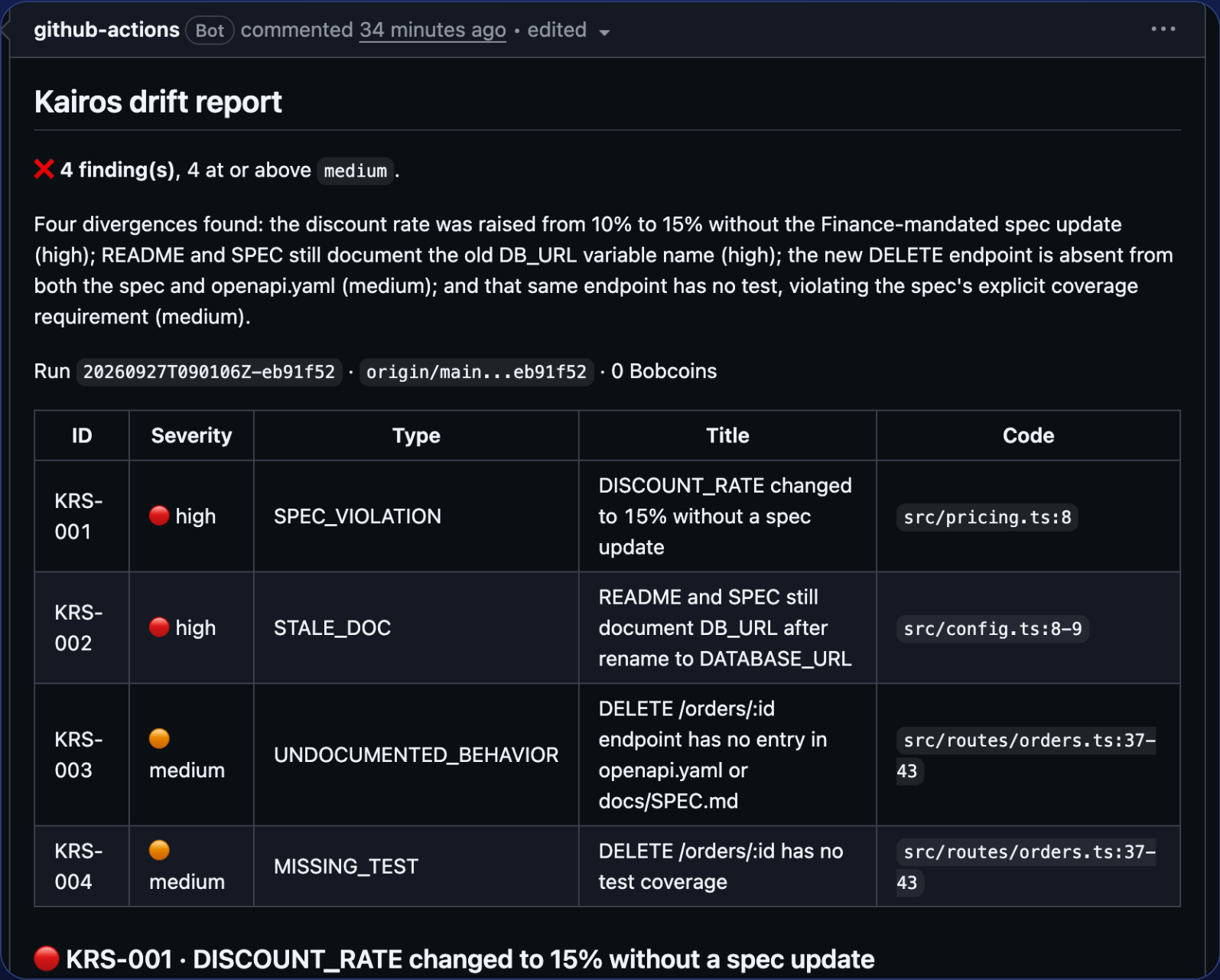
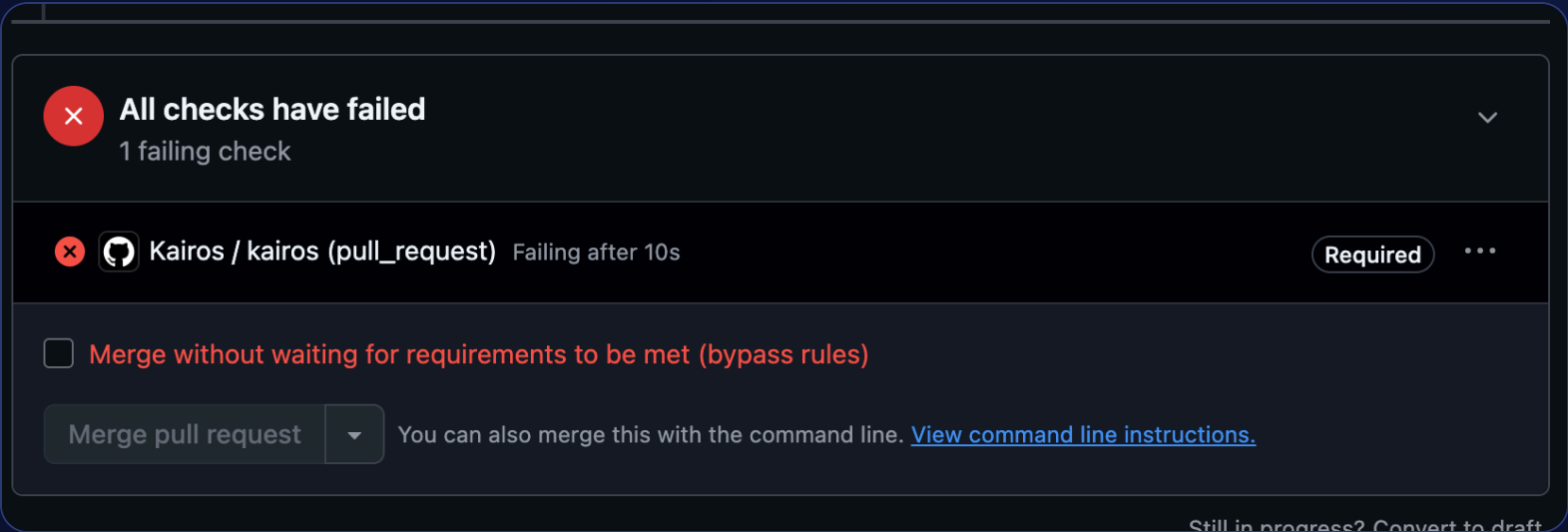
4 finding(s), 4 at or above medium. exit 1: push blocked
```

3/3 seeded drifts found, plus the missing test. **0 false positives** on a control commit (a doc typo fix).

On every pull request

GitHub Action · PR comment · required check

The same report lands on the PR as one comment, updated on every push. Mark the check as **required** and drift **blocks the merge**.



Fix: the right side of the truth

kairos fix --id <ID> --truth intent|code

C · env rename

The rename is intended →
docs follow the code. Bob
updates README and SPEC.

11 s · 0.058 Bobcoins

B · new endpoint

Intended → Bob **documents**
it in SPEC and OpenAPI and
writes 2 tests.

26 s · 0.138 Bobcoins

A · discount

Breaks a rule owned by
Finance → **the spec wins**;
Bob puts the code and tests
back to 10%.

28 s · 0.221 Bobcoins

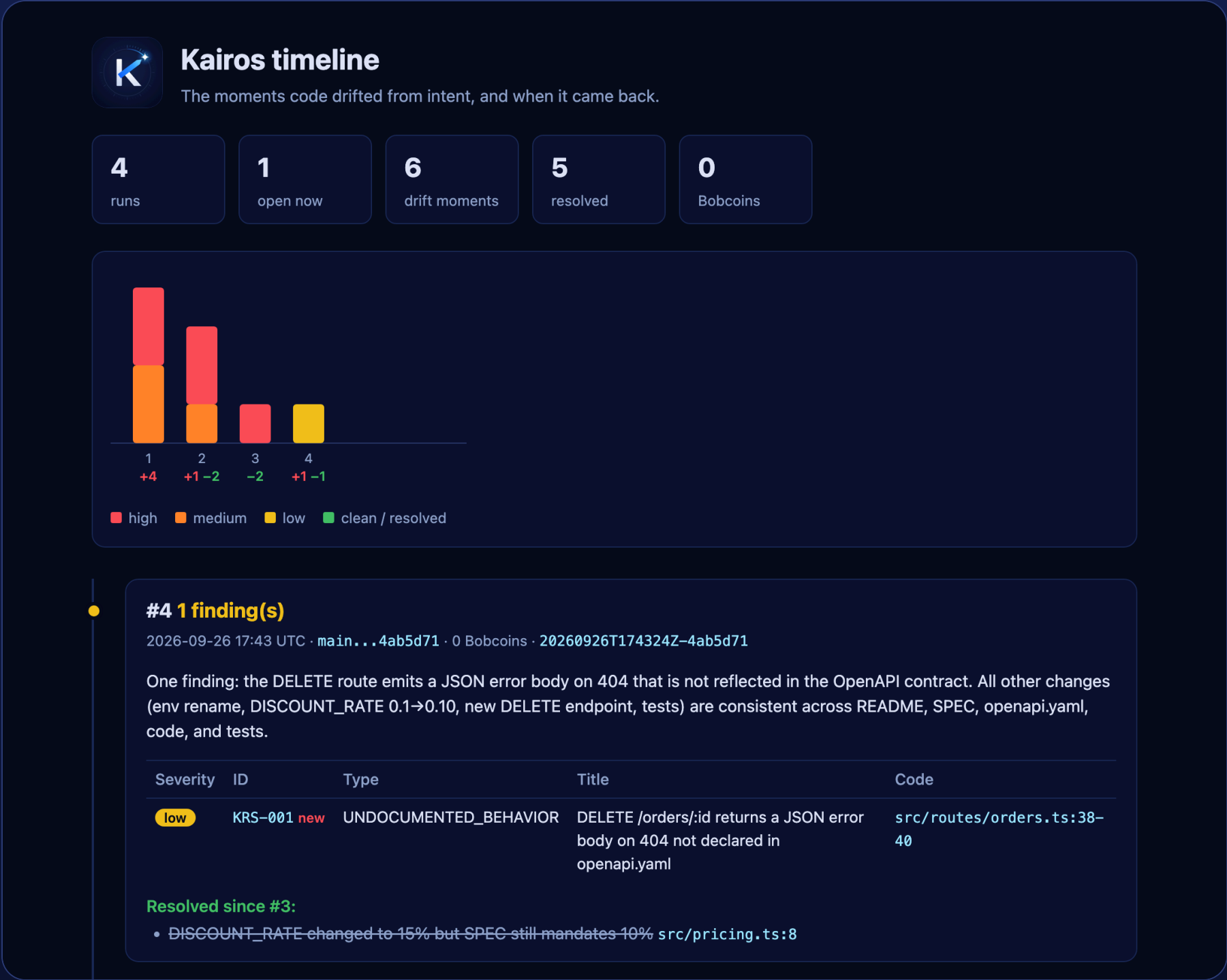
Each fix: Bob edits → you see the diff and confirm → Kairos commits, logs the decision in `DECISIONS.md`, re-checks. Docs-only by default; code only with `--allow-code`.

Every Kairos moment

kairos report --html

A single self-contained page: when drift appeared, when it was resolved, what it cost.

Last run: Bob noticed its own new endpoint returns a JSON 404 body the contract doesn't declare. **Low severity, below the threshold:** the check passes, the finding stays visible.



Living docs: continuity between sessions

kairos init · kairos-dev mode · kairos handoff

- `docs/kairos/`: SPEC, PLAN, PROGRESS, DECISIONS, one file per task, **HANDOFF**.
- Pointers in the Bob mode, `CLAUDE.md` and `AGENTS.md`: **every agent reads HANDOFF first**. A new chat starts with "continue".
- A post-commit hook logs progress; `kairos fix` logs decisions; `check` flags stale docs.
- Unlike context compaction: explicit, reviewed in git, shared by Bob, Claude Code and Codex.
- **Dogfooded**: Kairos was built this way. HANDOFF (3.8k chars) + one task file instead of ~70k chars of docs.

How IBM Bob powers it

- `bob run --mode kairos --format json`
with `--max-cost` and `--max-turns`
- **4 custom modes:** kairos (read-only),
kairos-fix (docs/tests via `fileRegex`),
kairos-fix-code, kairos-dev
- **Whole-repo understanding:** Bob opens
the code it needs (8 tool calls on A/B/C, 1
on the control)
- **Document understanding:** Markdown
specs, OpenAPI, READMEs, tests
- **Agent mode** for fixes; same path in CI via
the Action

On the demo, real Bob

Drifts caught / false positives	3/3 · 0
One check	20–35 s , 0.03–0.18 Bobcoins
Cached rerun	0 Bobcoins
Each fix	11–28 s
Check → 3 fixes → pass	5.5 min , 1.12 Bobcoins
Whole hackathon	4.85 / 40 Bobcoins

What's next

- **Nudges inside the IDE** at the moment of the edit, not only at push time.
- Fan a large check out to **Bob subagents**, one per intent area.
- More languages beyond JS/TS.
- An **org-wide drift dashboard**: which intents drift, where, how fast they come back.

Kairos · catch the moment code drifts from intent

github.com/OnixFireOne/ibm-kairos · MIT